

Kravspecifikation: LMR2CLI

Projektnamn: LMR2CLI (Language Model Runner to CLI)

Utvecklingsplattform: Rust (Latest Stable)

Målgrupp: Internt bruk / Kollegor (Cross-platform Mac/Windows)

Kontext: CLI-klient mot Mark Twain API

1. Inledning och Syfte

Detta dokument fungerar som en kravspecifikation för Claude Code (eller annan autonom agent) för att utveckla det fristående CLI-verktyget **LMR2CLI** i programmeringsspråket Rust. Verktyget ska agera som ett interaktivt och användarvänligt gränssnitt mot ett befintligt Mark Twain API, med fokus på minimala beroenden vid installationstillfället och en modern terminalbaserad användarupplevelse (UX).

2. Arkitektur och Projektstruktur

Projektet ska struktureras som ett standardsystem i Rust med Cargo som pakethanterare. Källkoden ska separeras i en logisk src-struktur.

```
lmr2cli/  
├── Cargo.toml  
├── README.md  
├── install.sh  
└── src/  
    ├── main.rs  
    ├── api.rs  
    ├── cli.rs  
    └── ui.rs
```

Beroenden (Dependencies)

Följande externa bibliotek (crates) bör användas för att uppnå önskad funktionalitet:

- **clap** (v4) - För parsning av kommandoradsargument och flaggor.
- **tokio** - För asynkron exekvering och API-anrop.
- **reqwest** - För HTTP-klientkommunikation med Mark Twain API.
- **serde / serde_json** - För serialisering och deserialisering av API-data.
- **dialoguer** eller liknande bibliotek - För interaktiva menyer, promptar och frågeguider.

3. Funktionella Krav och Kommandon

CLI-verktyget ska stödja direktkommandon (CLI-argument) samt ett interaktivt läge.

Kommando	Beskrivning	Exempel på indata / Flaggor
analyze-style	Analyserar stilen i en textsnutt mot Mark Twains språkliga profil.	--text "Inmatad text..." eller interaktiv prompt.
search	Utför en semantisk sökning i Mark Twain-databasen/korpusen.	--query "Sökord/Fras"
interactive	Startar ett guidat läge med menyer (motsvarande funktionalitet som Vercel/Gum).	Inga (standardläge om inga argument anges).

Felhantering och Avbrott: Programmet måste lyssna på Control+C för att avslutas omedelbart och graciöst, utan att lämna terminalen i ett korrupt tillstånd. I det interaktiva läget ska Escape eller ett explicit "Exit"-val också stänga applikationen.

4. UX/CLI Design och Visuellt Gränssnitt

- **Splash Screen / Välkomstbild:** Vid uppstart utan specifika flaggor (eller i interaktivt läge) ska en snygg ASCII-art-logotyp föreställande initialerna "MT" eller ett stiliserat ansikte visas.
- **Interaktiva element:** Använd färger (t.ex. via crossterm eller colored) och rullningslistor för val av kommandon så att användaren slipper memorera syntaxen.
- **Statusindikatorer:** Under API-anrop eller beräkningar ska en laddningsanimation (spinner) visas för att indikera pågående process.

5. Distribution och Installationsfas

Ett av de absolut viktigaste kraven är att slutanvändaren (kollegan) inte ska behöva installera Rust-miljön (Cargo/rustc) på sin maskin.

1. **Fristående Binärer:** Kompilering ska ske i release-läge (cargo build --release) för målplattformarna macOS (Intel/Apples Silicon) och Windows.
2. **Automatiskt Installationsskript (install.sh):** Ett skript ska skapas för att köras via curl. Skriptet ska:

- Känna av operativsystem (Mac eller Windows/Git Bash).
- Hämta rätt förkompilerad binär (zip/tar.gz) från GitHub Releases eller en distributionsserver.
- Packa upp binären och lägga den i användarens PATH (t.ex. /usr/local/bin eller motsvarande).

6. Instruktioner för Claude Code under Utveckling

När Claude Code initieras i det nya arkivet ska agenten följa dessa instruktioner:

- Generera en ren och modulär källkod i src/ med tydlig separering mellan API-logik och UI-komponenter.
- Konfigurera Cargo.toml med optimeringsflaggor för mindre binärstorlek (t.ex. lto = true, opt-level = "z").
- Skapa en utförlig README.md med instruktioner för hur man bygger projektet lokalt och hur man distribuerar de färdiga binärerna.